



Move Semantics and Smart Pointers

C++ Language

Move Semantics and Smart Pointers

- ❖ When we allocate memory dynamically we have to deallocate it properly
 - A dynamically allocated block of memory shall be released only once
 - In a complex situation releasing memory properly is a challenge
 - Even sometimes we forget to release dynamically allocated block of memories
- ❖ A class can help us to manage pointers and cleanup
 - The class destructor can be used to free memory automatically

```
template <typename T>
class AutoPtr {
    T *ptr;
public:
    AutoPtr(T *_ptr = nullptr) : ptr(_ptr) {}

    T &operator*() const { return *ptr; } // Overload the dereference operator
    T *operator->() const { return ptr; } // Overload the arrow operator

    ~AutoPtr() { delete ptr; }
};
```

```
#include <ctime>

void func(void) { if (0 == rand() % 2) { throw 0; } }

int main(void) {
    std::srand(std::time(NULL));
    try {
        int *a{new int{0}}; func(); delete a;
        int *b{new int{0}}; func(); delete b;
    } catch (...) {
        // What if one of the func calls throws an exception?
    }
    return 0;
}
```

Move Semantics and Smart Pointers

- ❖ In the example; a, b and c own the same pointer and when they are destroyed, the memory is released multiple times and the program crashes
 - To solve the problem C++ 98 introduced `std::auto_ptr` to implement moving ownership of dynamic objects by overloading a copy constructor and a copy assignment. `std::auto_ptr` was deprecated by C++11
- ❖ **Move semantics** means transferring ownership of an object rather than making a copy.
- ❖ To implement move semantics, C++ 11 introduced move constructors, move assignments and smart pointers
 - `std::unique_ptr`, `std::shared_ptr` and `std::weak_ptr`

```
#include <iostream>

template <typename T>
class AutoPtr { T *ptr;
public: AutoPtr(T *_ptr = nullptr) : ptr{_ptr} {}
    T &operator*() const { return *ptr; } // Overload the dereference operator
    T *operator->() const { return ptr; } // Overload the -> operator
    ~AutoPtr() { delete ptr; }
};

class Point { int x, y;
public:
    Point(int _x, int _y) : x{_x}, y{_y} { std::cout << "Point Created" << std::endl; }
    ~Point() { std::cout << "Point Destroyed" << std::endl; }
};

int main(void) {
    AutoPtr<Point> a{new Point{0, 0}};
    AutoPtr<Point> b{a}; // The copy constructor is called.
    AutoPtr<Point> c; c = a; // The copy assignment is called.
    return 0;
}
```

Move Semantics and Smart Pointers

- ❖ Copy semantics implemented using a copy constructor and a copy assignment
- ❖ We cannot create a function to return an AutoPtr. E.g. in the following function, we cannot return an instance or a reference/pointer of AutoPtr. Because in the end of the function the local instance is destroyed and the object is release.

```
??????? makeAutoPoint(int m, int n) {  
    return AutoPtr<Point>{new Point{m, n}};  
}  
  
AutoPtr<Point> a;  
a = makeAutoPoint(1, 2); //????
```

```
#include <iostream>  
  
template <typename T> class AutoPtr { T *ptr;  
public: AutoPtr(T *_ptr = nullptr) : ptr{_ptr} {}  
    // A copy constructor to implement move semantics  
    AutoPtr(AutoPtr &src) : ptr{src.ptr} /* transfer the ownership */  
    { src.ptr = nullptr; /* that no longer owns the pointer */ }  
    // An assignment operator that implements move semantics  
    AutoPtr &operator=(AutoPtr &src)  
    { if (this != &src) { delete ptr; ptr = src.ptr; src.ptr = nullptr; } return *this; }  
    T &operator*() const { return *ptr; } // Overload the dereference operator  
    T *operator->() const { return ptr; } // Overload the -> operator  
    ~AutoPtr() { delete ptr; }  
};  
  
class Point { int x, y; public:  
    Point(int _x, int _y) : x{_x}, y{_y} { std::cout << "Point Created" << std::endl; }  
    ~Point() { std::cout << "Point Destroyed" << std::endl; }  
};  
  
int main(void) {  
    AutoPtr<Point> a{new Point{0, 0}};  
    AutoPtr<Point> b{a}; // The copy constructor is called.  
    AutoPtr<Point> c; c = b; // The copy assignment is called.  
    return 0;  
}
```

Move Semantics and Smart Pointers

- ❖ If instead of **move semantics**, we implement **copy semantics** using the copy constructor and the copy assignment then we are able to create an `AutoPtr` instance using a function, but this solution is not efficient.
- ❖ In the example the point is copied twice which is not efficient.

```
AutoPtr<Point> makeAutoPoint(int m, int n) {  
    return AutoPtr<Point>{new Point{m, n}};  
}  
  
int main(void) {  
    AutoPtr<Point> c;  
    c = makeAutoPoint(1, 2); // The copy assignment is called.  
    AutoPtr<Point> d{c}; // The copy constructor is called.  
    return 0;  
}
```

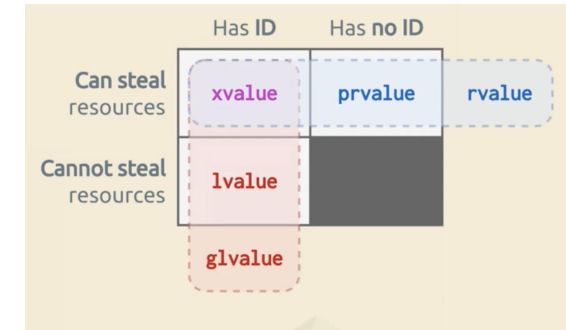
```
#include <iostream>  
  
template <typename T> class AutoPtr { T *ptr;  
public: AutoPtr(T *_ptr = nullptr) : ptr{_ptr} {}  
    // A copy constructor to implement copy semantics  
    AutoPtr(const AutoPtr &src) : ptr{new T{*src.ptr}} {} /* Deep copy */  
    // An assignment operator that implements copy semantics  
    AutoPtr &operator=(const AutoPtr &src)  
    { if (this != &src) {  
        delete ptr; /* Release any resource held by this instance */  
        ptr = new T{*src.ptr}; /* Deep copy */  
    } return *this;  
    }  
    T &operator*() const { return *ptr; } // Overload the dereference operator  
    T *operator->() const { return ptr; } // Overload the -> operator  
    ~AutoPtr() { delete ptr; }  
};  
  
class Point { int x, y; public:  
    Point(int _x = 0, int _y = 0) : x{_x}, y{_y} { std::cout << "Point Created" << std::endl; }  
    Point(const Point &point) : x{point.x}, y{point.y}  
    { std::cout << "Point Copied" << std::endl; }  
    ~Point() { std::cout << "Point Destroyed" << std::endl; }  
};
```

Move Semantics and Smart Pointers

❖ C++11 defines two new functions to implement move semantics

- A [move constructor](#) which has a parameter of type r-value reference
 - Implicitly is generated if there are no user-declared copy constructor, copy assignment operators, move assignment operators and destructor.
- A [move assignment](#) which has a parameter of type r-value reference
 - Implicitly is generated if there are no user-declared copy constructor, move constructor, copy assignment operators and destructor.
- Are called automatically if they have been defined.
- An r-value is typically a literal or a temporary value.
- C++ has a special rule which states that automatic objects returned from a function by value can be moved even if they are l-value expressions.
 - Possible to convert an l-value expression to an r-value reference. For example:

```
int a{10}; int &&ref{static_cast<int &&>(a)};
```



```
#include <iostream>

void func(int &lref) { std::cout << "l-value reference: " << lref << "\n"; }
void func(const int &lcref) { std::cout << "const l-value reference: " << lcref << "\n"; }
void func(int &&rref) { std::cout << "r-value reference: " << rref << "\n"; }
void func(const int &&rcref) { std::cout << "const r-value reference: " << rcref << "\n"; }

int main(void) {
    int x{5}; const int y{10};
    // If void func(int &rref) doesn't exist then void func(const int &lcref) will be called
    func(x); // l-value argument calls l-value version of function
    func(y); // l-value argument calls const l-value version of function
    // If void func(int &&rref) doesn't exist then void func(const int &lcref) will be called
    func(15); // r-value argument calls r-value version of function
    return 0;
}
```

Move Semantics and Smart Pointers

❖ Move constructor and move assignment

```
class Point {
    int x, y;
public:
    Point(int _x = 0, int _y = 0) : x{_x}, y{_y}
    { std::cout << "Point Created" << std::endl; }
    ~Point() { std::cout << "Point Destroyed" << std::endl; }
};

AutoPtr<Point> makeAutoPoint(int m, int n) {
    return AutoPtr<Point>{new Point{m, n}};
}

int main(void) {
    AutoPtr<Point> c;
    c = makeAutoPoint(1, 2); // The move assignment is called.
    auto d{static_cast<AutoPtr<Point>> &&>(c)}; // The move constructor is called.
    return 0;
}
```

```
#include <iostream>
template <typename T> class AutoPtr {
    T *ptr;
public:
    AutoPtr(T *_ptr = nullptr) : ptr{_ptr} {}
    AutoPtr(const AutoPtr &) = delete; // Copy constructor deleted
    AutoPtr &operator=(const AutoPtr &) = delete; // Copy assignment deleted

    // The move constructor to transfer ownership of src.ptr to ptr
    AutoPtr(AutoPtr &&src) noexcept : ptr{src.ptr} { src.ptr = nullptr; }
    // The move assignment to transfer ownership of src.ptr to ptr
    AutoPtr &operator=(AutoPtr &&src) noexcept {
        if (this != &src) {
            delete ptr; // Release any resource held by this instance
            ptr = src.ptr; // Transfer ownership
            src.ptr = nullptr;
        } return *this;
    }
    T &operator*() const { return *ptr; } // Overload the dereference operator
    T *operator->() const { return ptr; } // Overload the -> operator
    ~AutoPtr() { delete ptr; }
};
```

Move Semantics and Smart Pointers

- ❖ **std::move** is a standard function defined in **utility** that casts its argument to an r-value reference, so that move semantics can be invoked.

➤ E.g. `auto d{static_cast<AutoPtr<Point> &&>(c)};` could be written as `auto d{std::move(c)};`

```
#include <string>
#include <utility>
#include <iostream>
template <typename T> void swap1(T &a, T &b) {
    T tmp{a}; // invokes copy constructor
    a = b;    // invokes copy assignment
    b = tmp;  // invokes copy assignment
} template <typename T> void swap2(T &a, T &b) {
    T tmp{std::move(a)}; // invokes move constructor
    a = std::move(b);    // invokes move assignment
    b = std::move(tmp);  // invokes move assignment
} int main(void) { std::string x{"abc"}; std::string y{"de"};
    std::cout << "x: " << x << std::endl << "y: " << y << std::endl;
    swap1(x, y); swap2(x, y);
    std::cout << "x: " << x << std::endl << "y: " << y << std::endl;
    return 0; }
```

```
#include <string>
#include <vector>
#include <utility>
#include <iostream>

int main(void) {
    std::string str = "Hello"; std::vector<std::string> v;
    v.push_back(str); // uses push_back(const T&), copies str to the vector
    std::cout << "After copy, str is: " << str << std::endl;

    // uses push_back(T&&), str will be moved into the vector.
    // This is less expensive, but also means str might now be empty.
    v.push_back(std::move(str));
    std::cout << "After move, str is \"" << str << "\"" << std::endl;
    std::cout << "The contents of the vector are { " << v[0] << ", " << v[1] << " }";
    return 0;
}
```

Move Semantics and Smart Pointers

- ❖ `std::move_if_noexcept` returns an rvalue reference to its argument if the argument
 - Is move-only or its move constructor is `noexcept`
 - A move-only type has no copy constructor
 - Otherwise it returns an lvalue reference
- ❖ It is typically used to combine move semantics and strong exception guarantee.

```
int main(void) {  
    Resource a{13};  
    /* If the move constructor is noexcept then */  
    /* the move constructor is called; otherwise */  
    /* the copy constructor is used */  
    Resource b{std::move_if_noexcept(a)};  
    return 0;  
}
```

```
#include <utility>  
#include <iostream>  
  
#define NOEXCEPT /* Comment out this line */  
  
class Resource { int *ptr;  
public: Resource(int value) : ptr{new int{value}} {};  
    Resource(const Resource &src) { /* Copying using the copy constructor */  
        std::cout << "Copy constructor" << std::endl;  
        if (src.ptr != nullptr) { ptr = new int{*src.ptr}; }  
    }  
#ifdef NOEXCEPT  
    Resource(Resource &&src) noexcept : ptr{src.ptr} /* Move constructor */  
#else  
    Resource(Resource &&src) : ptr{src.ptr} /* Move constructor */  
#endif  
    { src.ptr = nullptr; std::cout << "Move constructor" << std::endl; }  
    ~Resource() { std::cout << "destroying " << *this << std::endl; delete ptr; }  
    friend std::ostream &operator<<(std::ostream &out, const Resource &res) {  
        out << "Resource(";  
        if (res.ptr == nullptr) { out << "empty"; } else { out << *res.ptr; }  
        out << ')'; return out; }  
};
```

Move Semantics and Smart Pointers

- ❖ `std::unique_ptr` is a smart pointer that owns and manages a dynamic object through a pointer and releases the object when the `unique_ptr` goes out of scope. `std::unique_ptr` should completely own the object it manages, not share the ownership with others.
- ❖ `std::make_unique` is a template function that constructs and initializes an object of any type with the arguments passed to the function.
- ❖ Use `std::make_unique` instead of creating `std::unique_ptr` and using `new` by yourself.
- ❖ E.g. `std::unique_ptr<Point> point{new Point{1, 2}};` could be written as `auto point{std::make_unique<Point>(10, 20)};`

```
#include <memory>
#include <iostream>

class Point { int x, y;
public:
    Point(int _x = 0, int _y = 0) : x{_x}, y{_y}
    { std::cout << "Point Created" << std::endl; }
    friend std::ostream &operator<<(std::ostream &out, const Point &p)
    { out << "(" << p.x << ", " << p.y << ")"; return out; }
    ~Point() { std::cout << "Point Destroyed" << std::endl; }
};

void print(const Point *ptr) { if (ptr != nullptr) { std::cout << *ptr << std::endl; } }

int main(void) { std::unique_ptr<Point> point{new Point{1, 2}};
    print(point.get()); /* get a pointer to the Point */
    return 0;
}
```

Move Semantics and Smart Pointers

- ❖ `std::shared_ptr` is a smart pointer that retains shared ownership of an object through a pointer. Several `shared_ptr` objects may own the same object. The object is destroyed and the memory is deallocated when the last remaining `shared_ptr` owning the object is destroyed or another pointer is assigned to the last remaining `shared_ptr` owning the object.
- ❖ `std::make_shared` can be used to create a `std::shared_ptr` for a dynamic object. E.g.

```
auto ptr{std::make_shared<Point>(1, 2)};
```
- ❖ It is possible to convert a `std::unique_ptr` to a `std::shared_ptr`. For example:

```
std::shared_ptr<Point> ptr{std::make_unique<Point>(1, 2)};
```

```
#include <memory>
#include <iostream>

class Point { int x, y;
public:
    Point(int _x = 0, int _y = 0) : x{_x}, y{_y}
    { std::cout << "Point Created" << std::endl; }
    friend std::ostream &operator<<(std::ostream &out, const Point &p)
    { out << "(" << p.x << ", " << p.y << ")"; return out; }
    ~Point() { std::cout << "Point Destroyed" << std::endl; }
};

void print(const Point *ptr) { if (ptr != nullptr) { std::cout << *ptr << std::endl; } }

int main(void) { std::shared_ptr<Point> ptr{new Point{1, 2}};
    { std::shared_ptr<Point> tmp{ptr};
        /* tmp gets out of the scope and destroyed */ }
    print(ptr.get()); return 0;
}
```

Move Semantics and Smart Pointers

- ❖ `std::weak_ptr` is a smart pointer that holds a non-owning ("weak") reference to an object that is managed by `std::shared_ptr`. It must be converted to `std::shared_ptr` in order to access the referenced object. `std::weak_ptr` models temporary ownership when:
 - An object needs to only be accessed if it exists, `std::weak_ptr` is used to track the object
 - The object may be deleted at any time by someone else.
 - The `lock()` function shall be used to convert a `std::weak_ptr` to a `std::shared_ptr` in order to get access to the object. If the original `std::shared_ptr` is destroyed while a `std::weak_ptr` has locked the object, lifetime of the object is extended until the temporary `std::shared_ptr`(locked by a `std::weak_ptr`) is destroyed as well.
 - A `std::weak_ptr` is used to break reference cycles formed by objects managed by `std::shared_ptr`.
- ❖ The `use_count()` function returns the number of `shared_ptr` objects that manage the object
- ❖ The `reset()` function releases the ownership of the managed object

Move Semantics and Smart Pointers

```
#include <string>
#include <memory>
#include <iostream>

class Person { std::string name;
    std::weak_ptr<Person> partner; // If it is a std::shared_ptr then objects are not destroyed.
public:
    Person(const std::string &_name) : name(_name) { std::cout << name << " created" << std::endl; }
    ~Person() { std::cout << name << " destroyed" << std::endl; }
    const std::shared_ptr<Person> getPartner(void) const { return partner.lock(); }
    const std::string &getName() const { return name; }
    friend void partnership(std::shared_ptr<Person> &p1, std::shared_ptr<Person> &p2) {
        p1->partner = p2; p2->partner = p1;
        std::cout << p1->name << " is now the partner of " << p2->name << std::endl;
    }
};

int main(void) {
    auto lucy{std::make_shared<Person>("Lucy")}; auto ricky{std::make_shared<Person>("Ricky")};
    partnership(lucy, ricky); auto partner = ricky->getPartner(); // get a shared_ptr to Ricky's partner
    std::cout << ricky->getName() << "'s partner is: " << partner->getName() << std::endl;
    return 0;
}
```

```
#include <memory>
#include <iostream>

void observe(std::weak_ptr<int> &wkptr) {
    std::cout << "count = " << wkptr.use_count() << "; ";
    // Make a copy of the shared pointer
    std::shared_ptr<int> spt{wkptr.lock()};
    if (spt != nullptr)
        { std::cout << "*spt == " << *spt << std::endl; }
    else { std::cout << "wkptr is expired" << std::endl; }
}

int main(void) {
    std::weak_ptr<int> tmp;
    {
        auto sp{std::make_shared<int>(42)};
        tmp = sp; observe(tmp);
    }
    observe(tmp);
    return 0;
}
```

C++ Language

❖ Some useful links

- [C++ Reference](#)
- [Standard C++ Library Reference](#)
- [C++ Tutorial](#)
- [C++ Language](#)
- [C++ Tutorial](#)
- [C++ Full Course For Beginners \(Learn C++ in 10 hours\)](#)
- [C++ Tutorial for Beginners - Full Course](#)
- [C++ Tutorial 2021](#)
- [C++ Programming Tutorials Playlist](#)